

Optimization for Training Deep Models

Gwangsu Kim

JBNU Dep. of Stat.

First Semester 2025

Special Topics for Machine Learning

Outline

Intro.

Challenges in Neural Network Optimizations

Basic Algorithms

Algorithms with Adaptive Learning Rates & Others

Optimization Strategies and Meta-Algorithms

Intro. I

1. Intro.

- ▶ Optimization is to find the (local) minimum of the loss function.
- ▶ The ideal object function is

$$J^*(\theta) = \mathbb{E}_{(\mathbf{x}, y) \sim p_{data}}[J(f(\mathbf{x} \mid \theta), y)],$$

averaged on sample space.

- ▶ In practice, we use

$$J^*(\theta) = \mathbb{E}_{(\mathbf{x}, y) \sim \hat{p}_{data}}[J(f(\mathbf{x} \mid \theta), y)] = \sum_{i=1}^N J(f(x_i \mid \theta), y_i) / N.$$

Intro. II

- ▶ ERM (empirical risk minimization): using finite samples.
 1. Prone to overfitting.
 2. Infeasible in the calculation.
 3. Non-differentiable loss function.
- ▶ Surrogate loss function and early stopping
 1. For example, in binary classification, $\mathbb{I}(y \neq f)$ is replaced by $\frac{\exp(yf)}{1+\exp(yf)}$. Note that the later loss can decrease, although the former loss is 0.
 2. The (surrogate) loss and prediction error (original loss) cannot be the same. Minimizing loss can help to achieve a more robust prediction.
 3. Usually, the early stopping rule uses the prediction error. Therefore, the (surrogate) loss can have a somewhat larger value.

Intro. III

- ▶ Batch and minibatch algorithm.
 1. Optimization algorithms using the entire training set are batch or deterministic gradient methods.
 2. Optimization algorithms that use only one example at a time are sometimes called stochastic and sometimes on-line methods.
 3. Using more than one but fewer than all the training examples, minibatch or minibatch stochastic algorithms.
- ▶ On-line type algorithm can use parallel computing by GPU.
 - Critical points: local minima, local maxima, and saddle point.

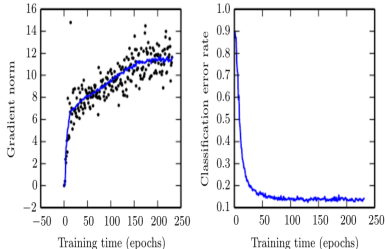
Challenges in neural network optimizations I

1. Ill-conditioning

- ▶ Second-order Taylor series for the direction to $\epsilon \mathbf{g}$.

$$\frac{1}{2} \epsilon^2 \mathbf{g}^\top H \mathbf{g} - \epsilon \mathbf{g}^\top \mathbf{g}$$

- ▶ The $\epsilon^2 \mathbf{g}^\top H \mathbf{g}$ can be dominant to $\epsilon \mathbf{g}^\top \mathbf{g}$.



- ▶ Newton-Raphson requires heavy modification in the use of deep networks.

Challenges in neural network optimizations II

2. Local Minima

- ▶ Basically, neural networks cannot provide the unique solution for the minimum loss (non-identifiable) because of the invariant w.r.t. location of layers (weight symmetry), filters in CNN, and others.
- ▶ Local minima problem is usually observed. But experts now suspect that, for sufficiently large neural networks, most local minima have a low-cost function value and that it is not important to find a truly global minimum rather than to find a point in parameter space that has a low but not minimal cost.
- ▶ Gradient norm can be a criterion to understand learning behavior.

Challenges in neural network optimizations III

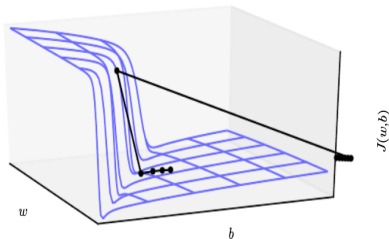
3. Plateaus, Saddle Points, and Other Flat Regions

- ▶ Saddle points: gradient is 0. However, there exist directions for minimizing and maximizing the loss.
- ▶ In random functions, there are many saddle points in a high dimensional space, compared to many local minima in a low dimensional space. Similar observed in neural networks.
- ▶ Implications: 1) High loss values and saddle points - quickly escape by SGD, 2) high-cost plateaus can be problematic. However, the global minima can be achieved in plateaus.
- ▶ Gradient 0: local minima, local maxima, and saddle points.

Challenges in neural network optimizations IV

4. Cliffs and Exploding Gradients

- ▶ Rapidly larger gradient (extremely steep can cause a strong jump (dangerous). Clipping can be helpful.



Challenges in neural network optimizations V

5. Long-Term Dependency

- ▶ In deep architecture such as RNN, we can have iterated form of $\mathbf{W}^t$.

$$\mathbf{W}^t = \mathbf{V} \text{diag}(\boldsymbol{\lambda})^t \mathbf{V}^{-1},$$

where $\mathbf{W} = \mathbf{V} \mathbf{V}^{-1}$.

- ▶ When $|\lambda_i| < 1$, the weight can banishing.

Challenges in neural network optimizations VI

6. Poor Correspondence between Local and Global Structure

- ▶ Bad initialization cannot ensure the (local/global) minima.
- ▶ It requires a long trajectory for the (local/global) minima.
- If there exists a region of space connected reasonably directly to a solution by a path that local descent can follow, and if we can initialize learning within that well-behaved region, we can avoid the almost issues.

Challenges in neural network optimizations VII

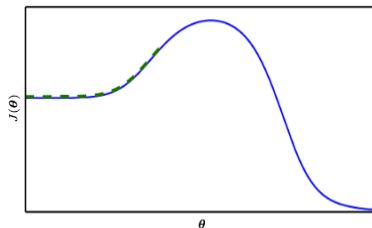


Figure 8.4: Optimization based on local downhill moves can fail if the local surface does not point toward the global solution. Here we provide an example of how this can occur, even if there are no saddle points or local minima. This example cost function contains only asymptotes toward low values, not minima. The main cause of difficulty in this case is being initialized on the wrong side of the “mountain” and not being able to traverse it. In higher-dimensional space, learning algorithms can often circumnavigate such mountains, but the trajectory associated with doing so may be long and result in excessive training time, as illustrated in figure 8.2.

Basic Algorithms I

1. Stochastic Gradient Descent

- ▶ Mostly used. The SGD gradient estimator introduces a source of noise (the random sampling of m training examples) that does not vanish even when we arrive at a minimum.

Algorithm 8.1 Stochastic gradient descent (SGD) update

Require: Learning rate schedule $\epsilon_1, \epsilon_2, \dots$

Require: Initial parameter θ

$k \leftarrow 1$

while stopping criterion not met **do**

 Sample a minibatch of m examples from the training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with corresponding targets $\mathbf{y}^{(i)}$.

 Compute gradient estimate: $\hat{\mathbf{g}} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$

 Apply update: $\theta \leftarrow \theta - \epsilon_k \hat{\mathbf{g}}$

$k \leftarrow k + 1$

end while

* Notations: $\mathbf{x}^{(i)} \equiv \mathbf{x}_i$, $L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)}) \equiv J(f(\mathbf{x}_i | \theta), y_i)$.

Basic Algorithms II

- ▶ A sufficient condition for the guarantee of the convergence of SGD.

$$\sum_{k=1}^{\infty} \epsilon_k = \infty$$
$$\sum_{k=1}^{\infty} \epsilon_k^2 < \infty$$

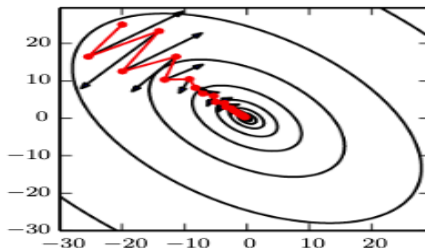
- ▶ Learning rate.

$$\epsilon_k = (1 - \alpha)\epsilon_0 + \alpha\epsilon_{\tau}, \quad \alpha = k/\tau, \quad k = 1, \dots, \tau.$$

- ▶ Theoretical results with an assumption of convexity: $J(\theta) - \arg \min_{\theta} J(\theta)$ is $O(1/\sqrt{k})$
- ▶ Also, $J(\theta) - \arg \min_{\theta} J(\theta)$ is $O(1/k)$ with a strong convexity.

Basic Algorithms III

2. Momentum



- The name momentum derives from a physical analogy, in which the negative gradient is a force moving a particle through parameter space, according to Newton's laws of motion.

Basic Algorithms IV

- Update rule.

$$\begin{aligned}\mathbf{v} &\leftarrow \alpha \mathbf{v} - \epsilon \nabla_{\boldsymbol{\theta}} \left(\frac{1}{m} \sum_{i=1}^m J(f(\mathbf{x}_i), y_i \mid \boldsymbol{\theta}) \right) \\ \boldsymbol{\theta} &\leftarrow \boldsymbol{\theta} + \mathbf{v}\end{aligned}$$

- Note

1. If $\mathbf{g} = \nabla_{\boldsymbol{\theta}} \left(\frac{1}{m} \sum_{i=1}^m J(f(\mathbf{x}_i), y_i \mid \boldsymbol{\theta}) \right)$ is identical in the long term, then the step size is

$$\frac{\epsilon}{(1 - \alpha) \|\mathbf{g}\|}.$$

2. Common values of α used in practice include 0.5, 0.9, and 0.99.

Basic Algorithms V

- ▶ We can view the momentum algorithm as simulating a particle subject to continuous-time Newtonian dynamics.

1. Velocity $\mathbf{v}(t)$ and net force $\mathbf{f}(t)$.

$$\mathbf{v}(t) = \frac{\partial}{\partial t} \theta(t)$$

$$\mathbf{f}(t) = \frac{\partial}{\partial t} \mathbf{v}(t)$$

2. Gradient and negative direction are combined in solving the differential equation.

Basic Algorithms VI

3. Nesterov Momentum

- ▶ Updating rule.

$$\begin{aligned}\mathbf{v} &\leftarrow \alpha \mathbf{v} - \epsilon \nabla_{\boldsymbol{\theta}} \left(\frac{1}{m} \sum_{i=1}^m J(f(\mathbf{x}_i), y_i \mid \boldsymbol{\theta} - \alpha \mathbf{v}) \right) \\ \boldsymbol{\theta} &\leftarrow \boldsymbol{\theta} + \mathbf{v}\end{aligned}$$

- ▶ Evaluation of gradient at the preceding point.

4. Initialization.

- ▶ Issues.
 1. Stability, final costs
 2. Generalization, ...

Basic Algorithms VII

- ▶ Perhaps the only property known with complete certainty is that the initial parameters need to "break symmetry" between different units.
 - The same initialized values can make the update the same. No gradient patterns are lost.
- ▶ Random initialization for weights: It is the direct way to address "break symmetry". Indep. r.v. for parameters or orthogonal random matrices.

Basic Algorithms VIII

1. In both, the scale is the most influential factor: Large values achieve "break symmetry" quickly. However, gradient explosion can be observed.
2. With a fully connected layer m inputs and n outputs, Glorot and Benjio (2010) suggest the normalization initialization such as

$$\text{Unif} \left[-\sqrt{\frac{6}{m+n}}, \sqrt{\frac{6}{m+n}} \right],$$

compromising between the goal of initializing all layers to have the same activation variance and the goal of initializing all layers to have the same gradient variance.

Basic Algorithms IX

- ▶ Optimization and regularization
 1. Optimization requires a relatively larger norm, and regularization makes the normal smaller.
 2. Ideally, we have small norm initial values, and a small update can achieve high performance.
 3. There are studies claiming that only initialization can achieve high performance.
- ▶ Not optimal performance.
 1. The norm-preserving can not be essential.
 2. Properties are not persistent during the training phase.
 3. Only speeding optimization, not strengthening the generalization.

Basic Algorithms X

- ▶ Bias and others: relatively simple.
 1. The zero initialization and some exceptions: 1) prior information, 2) when 0 can make many saturated activation outputs, and 3) losing the chance of learning for other parameters.
 2. For variance parameters, usually one can be used.
- The learning the better initialization can be (on-going) future works.

Algorithms with Adaptive Learning Rates & Others I

1. AdaGrad and RMSProp

- ▶ A larger gradient axis can be degraded at a speed.

Algorithm 8.4 The AdaGrad algorithm

Require: Global learning rate ϵ

Require: Initial parameter θ

Require: Small constant δ , perhaps 10^{-7} , for numerical stability

Initialize gradient accumulation variable $\mathbf{r} = \mathbf{0}$

while stopping criterion not met **do**

 Sample a minibatch of m examples from the training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with corresponding targets $\mathbf{y}^{(i)}$.

 Compute gradient: $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$.

 Accumulate squared gradient: $\mathbf{r} \leftarrow \mathbf{r} + \mathbf{g} \odot \mathbf{g}$.

 Compute update: $\Delta \theta \leftarrow -\frac{\epsilon}{\delta + \sqrt{\mathbf{r}}} \odot \mathbf{g}$. (Division and square root applied element-wise)

 Apply update: $\theta \leftarrow \theta + \Delta \theta$.

end while

Algorithms with Adaptive Learning Rates & Others II

- Avoiding the problem of specific learning rate is 0.

Algorithm 8.5 The RMSProp algorithm

Require: Global learning rate ϵ , decay rate ρ

Require: Initial parameter θ

Require: Small constant δ , usually 10^{-6} , used to stabilize division by small numbers

Initialize accumulation variables $\mathbf{r} = 0$

while stopping criterion not met **do**

 Sample a minibatch of m examples from the training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with corresponding targets $\mathbf{y}^{(i)}$.

 Compute gradient: $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$.

 Accumulate squared gradient: $\mathbf{r} \leftarrow \rho \mathbf{r} + (1 - \rho) \mathbf{g} \odot \mathbf{g}$.

 Compute parameter update: $\Delta \theta = -\frac{\epsilon}{\sqrt{\delta + \mathbf{r}}} \odot \mathbf{g}$. ($\frac{1}{\sqrt{\delta + \mathbf{r}}}$ applied element-wise)

 Apply update: $\theta \leftarrow \theta + \Delta \theta$.

end while

Algorithms with Adaptive Learning Rates & Others III

2. Adam (adaptive momentum)

- Incorporate the momentum and adaptive learning rate. Bias correction term.

Algorithm 8.7 The Adam algorithm

Require: Step size ϵ (Suggested default: 0.001)

Require: Exponential decay rates for moment estimates, ρ_1 and ρ_2 in $[0, 1)$.
(Suggested defaults: 0.9 and 0.999 respectively)

Require: Small constant δ used for numerical stabilization (Suggested default: 10^{-8})

Require: Initial parameters θ

Initialize 1st and 2nd moment variables $\mathbf{s} = \mathbf{0}$, $\mathbf{r} = \mathbf{0}$

Initialize time step $t = 0$

while stopping criterion not met **do**

 Sample a minibatch of m examples from the training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with corresponding targets $\mathbf{y}^{(i)}$.

 Compute gradient: $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$

$t \leftarrow t + 1$

 Update biased first moment estimate: $\mathbf{s} \leftarrow \rho_1 \mathbf{s} + (1 - \rho_1) \mathbf{g}$

 Update biased second moment estimate: $\mathbf{r} \leftarrow \rho_2 \mathbf{r} + (1 - \rho_2) \mathbf{g} \odot \mathbf{g}$

 Correct bias in first moment: $\hat{\mathbf{s}} \leftarrow \frac{\mathbf{s}}{1 - \rho_1^t}$

 Correct bias in second moment: $\hat{\mathbf{r}} \leftarrow \frac{\mathbf{r}}{1 - \rho_2^t}$

 Compute update: $\Delta \theta = -\epsilon \frac{\hat{\mathbf{s}}}{\sqrt{\hat{\mathbf{r}} + \delta}}$ (operations applied element-wise)

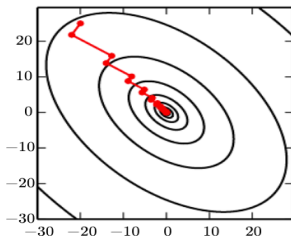
 Apply update: $\theta \leftarrow \theta + \Delta \theta$

end while

Algorithms with Adaptive Learning Rates & Others IV

3. Approximate the Second-order Methods

- ▶ Newton's method: $\theta^* = \theta_0 - H^{-1} \nabla_{\theta} J(\theta_0)$. Using Hessian (intractable).
- ▶ Conjugate gradient: $\mathbf{d}_{t-1}^T \mathbf{d}_t = 0$



Algorithms with Adaptive Learning Rates & Others V

- Considering the orthogonal direction of gradient updating.

$$\mathbf{d}_t = \nabla_{\theta} J(\theta) + \beta_t \mathbf{d}_{t-1},$$

where $\mathbf{d}_{t-1}^T \mathbf{d}_t = 0$.

1. Fletcher-Reeves

$$\frac{\nabla_{\theta} J(\theta_t)^T \nabla_{\theta} J(\theta_t)}{\nabla_{\theta} J(\theta_{t-1})^T \nabla_{\theta} J(\theta_{t-1})}$$

2. Polak-Ribière

$$\frac{(\nabla_{\theta} J(\theta_t) - \nabla_{\theta} J(\theta_{t-1}))^T \nabla_{\theta} J(\theta_t)}{\nabla_{\theta} J(\theta_{t-1})^T \nabla_{\theta} J(\theta_{t-1})}$$

Algorithms with Adaptive Learning Rates & Others VI

Algorithm 8.9 The conjugate gradient method

Require: Initial parameters θ_0

Require: Training set of m examples

Initialize $\rho_0 = \mathbf{0}$

Initialize $g_0 = 0$

Initialize $t = 1$

while stopping criterion not met **do**

 Initialize the gradient $g_t = \mathbf{0}$

 Compute gradient: $g_t \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$

 Compute $\beta_t = \frac{(g_t - g_{t-1})^\top g_t}{g_{t-1}^\top g_{t-1}}$ (Polak-Ribière)

 (Nonlinear conjugate gradient: optionally reset β_t to zero, for example if t is a multiple of some constant k , such as $k = 5$)

 Compute search direction: $\rho_t = -g_t + \beta_t \rho_{t-1}$

 Perform line search to find: $\epsilon^* = \operatorname{argmin}_{\epsilon} \frac{1}{m} \sum_{i=1}^m L(f(\mathbf{x}^{(i)}; \theta_t + \epsilon \rho_t), \mathbf{y}^{(i)})$

 (On a truly quadratic cost function, analytically solve for ϵ^* rather than explicitly searching for it)

 Apply update: $\theta_{t+1} = \theta_t + \epsilon^* \rho_t$

$t \leftarrow t + 1$

end while

Algorithms with Adaptive Learning Rates & Others VII

- Broyden–Fletcher–Goldfarb–Shanno (BFGS) algorithm.

Using the approximation of $\mathbf{H}^{-1}$, $\mathbf{M}$.

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + \epsilon \boldsymbol{\rho}_t,$$

where $\boldsymbol{\rho}_t = -\mathbf{M} \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_t)$.

Optimization Strategies and Meta-Algorithms I

1. Batch Normalization

- ▶ Simple example with a linear model.

$$\hat{y} = xw_1 \cdots w_l,$$

where $h_i = h_{i-1}w_i$.

1. $\mathbf{g} = \nabla_{\mathbf{w}} \hat{y}$ and $\mathbf{w} = \mathbf{w} - \epsilon \mathbf{g}$.
2. After updating, $\hat{y}$ is

$$x(w_1 - \epsilon g_1) \cdots (w_l - \epsilon g_l)$$

3. One of the second order term: $\epsilon^2 g_1 g_2 \prod_{i=3}^l w_i$. In this case, this term can be larger or smaller.
- ▶ Selection of learning rate is difficult, and high-order alg. is not feasible in deep networks.

Optimization Strategies and Meta-Algorithms II

- ▶ The parametrization (by batch normalization, normalizing h_i) significantly reduces the problem of coordinating updates across many layers.
- ▶ Let $\mathbf{H}$ be a mini-batch of layer activations to normalize.

$$\mathbf{H}' = \frac{\mathbf{H} - \boldsymbol{\mu}}{\sigma},$$

where $\boldsymbol{\mu} = \frac{1}{m} \sum_{i=1}^m \mathbf{H}_{i,:}$, and $\sigma = \sqrt{\frac{1}{m} \sum_{i=1}^m (\mathbf{H} - \boldsymbol{\mu})_i^2}$. In algorithms, all are back-propagated.

- Penalization does not work well, and normalization at each iteration requires too expensive computational costs.

Optimization Strategies and Meta-Algorithms III

- ▶ In a linear model, normalization is too strong restriction since the variation can be information. However, in deep networks, the non-linear activation exists.
- ▶ Normalizing the mean and standard deviation of a unit can reduce the expressive power of the neural network containing that unit.
- ▶ Learning the mean and variance for the high performance: $\gamma H' + \beta$
 - In the prediction (inference), the μ and σ uses the moving average in training epochs.

Optimization Strategies and Meta-Algorithms IV

2. Coordinate Descent and Polyak Averaging

- ▶ The (block) coordinate algorithm in a penalty function: Regularization of each coordinate given others are fixed.
- ▶ The basic idea of Polyak averaging: The optimization algorithm may leap back and forth across a valley several times without ever visiting a point near the bottom of the valley.

$$\theta^{(t)} = \alpha \theta^{(t-1)} + (1 - \alpha) \theta^{(t)}$$

Optimization Strategies and Meta-Algorithms V

3. Supervised Pretraining

- ▶ Greedy alg.: Break a problem into many components, then solve for the optimal version of each component in isolation. The combination does not ensure the optimality.
- ▶ Fine-tuning: Using the trained weights as the initial weights and many versions exist (freezing/non-freezing).
- ▶ Greedy supervised pretraining: Layer extension (pre-training small layers and using the weights for these layers and learning other layers in deeper networks; plug-in the prediction of the last layer as the input of the new layers).

Optimization Strategies and Meta-Algorithms VI

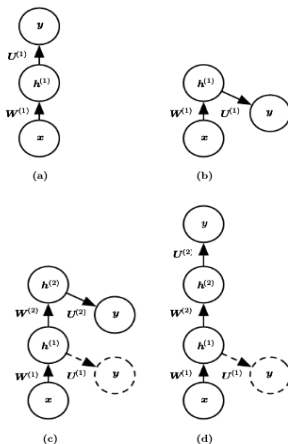


Figure 8.7: Illustration of one form of greedy supervised pretraining (Bengio et al., 2007). (a) We start by training a sufficiently shallow architecture. (b) Another drawing of the same architecture. (c) We keep only the input-to-hidden layer of the original network and discard the hidden-to-output layer. We send the output of the first hidden layer as input to another supervised single hidden layer MLP that is trained with the same objective as the first network, thus adding a second hidden layer. This can be repeated for as many layers as desired. (d) Another drawing of the result, viewed as a feedforward network. To further improve the optimization, we can jointly fine-tune all the layers, either only at the end or at each stage of this process.

Optimization Strategies and Meta-Algorithms VII

- ▶ Transfer learning: Similar to fine-tuning. But pre-trained on a different task and dataset.
- ▶ FitNet (teacher and student): Making a student network having the same (similar) prediction to a teacher network in some layers or final result.

Optimization Strategies and Meta-Algorithms VIII

4. Continuation Methods and Curriculum Learning

- ▶ Idea: Construction of sequential object function $\{J^{(k)}(\theta)\}_{k=0}^n$ for the same task.
- ▶ From fairly easy to difficult object functions. Learning from an object function and using the estimates for the initial values for the next learning (incremental harder).
- ▶ Continuation methods, traditionally, were mostly designed to overcome the challenge of local minima. Blurring operation is like as

$$J^{(i)}(\theta) = \mathbb{E}_{\theta' \sim \mathcal{N}(\theta' | \theta, \sigma^{(i)2})} [J(\theta)] .$$

- The intuition for this approach is that some non-convex functions become approximately convex when blurred.

Optimization Strategies and Meta-Algorithms IX

- ▶ Can break down:
 1. Costs are high to make an effective spectrum.
 2. Non-convex with any blurring.
 3. Blurring can lead to local rather than global minimum.
- ▶ The easier objective functions introduced by the continuation method can eliminate flat regions, decrease variance in gradient estimates, improve conditioning of the Hessian matrix, or do anything else that will either make local updates easier to compute or improve the correspondence between local update directions and progress toward a global solution

Optimization Strategies and Meta-Algorithms X

- ▶ Curriculum learning: Better results could be obtained by following a curriculum on a large-scale neural language modeling task. Curriculum learning has been successful in a wide range of natural languages.
- ▶ Stochastic curriculum, in which a random mix of easy and difficult examples is always presented to the learner, but where the average proportion of the more difficult examples (here, those with longer-term dependencies) is gradually increased.